

Diagrama de Sequência

O Diagrama de Sequência tem como objetivo mostrar como as mensagens entre os objetos são trocadas no decorrer do tempo para a realização de uma operação.

Ele é construído a partir do Diagrama de Casos de Uso, onde primeiro define-se qual o papel do sistema (casos de uso) e, em seguida, como o software realizará seu papel (sequência de operações).

O Diagrama de Sequência dá destaque a ordem temporal em que as mensagens são trocadas entre os objetos e atores de um sistema. Entende-se por mensagens os serviços solicitados de um objeto a outro e as respostas desenvolvidas para as solicitações.

Quando realiza-se a programação Orientada a Objetos (OO) precisaremos desenvolver antes o Diagrama de Classes, para saber quais objetos vamos precisar e os métodos que serão disparados no diagrama.

Os Diagramas de Sequência apresentam os seguintes elementos:

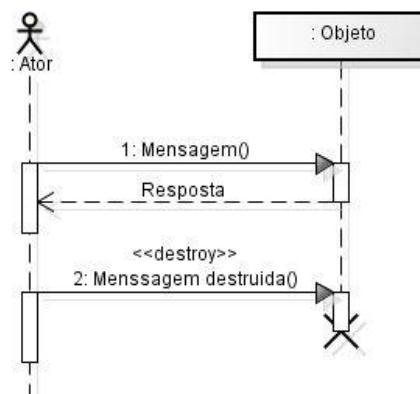


Atores: são entidades externas que interagem com o sistema e que solicitam serviços.

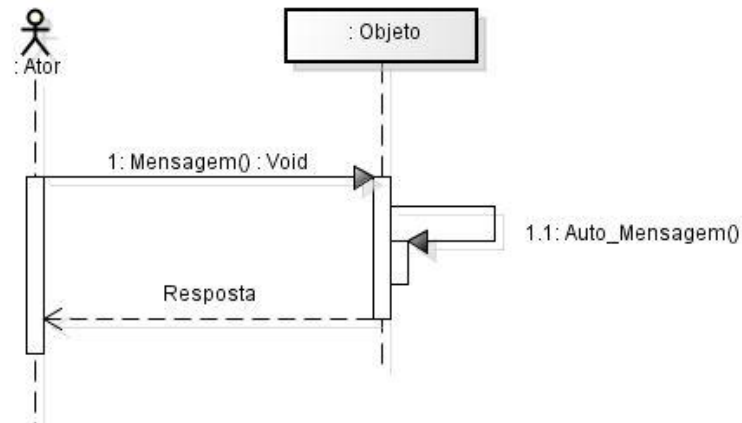
Linha de Vida (lifeline): linhas verticais tracejadas que representam o tempo de vida de um objeto.

Objetos: representam as instâncias das classes no processo.

Essas linhas verticais são preenchidas por barras também verticais que indicam exatamente quando um objeto existiu. Quando ele é destruído, é inserido um "X" na parte inferior da barra.



Mensagens: linhas horizontais representam mensagens trocadas entre objetos. Essas linhas são acompanhadas de um rótulo que contém o nome da mensagem e, opcionalmente, seus parâmetros. Observe que também podem existir mensagens enviadas para o mesmo objeto representando uma auto chamada. Mensagens de retorno são representadas por linhas horizontais tracejadas.

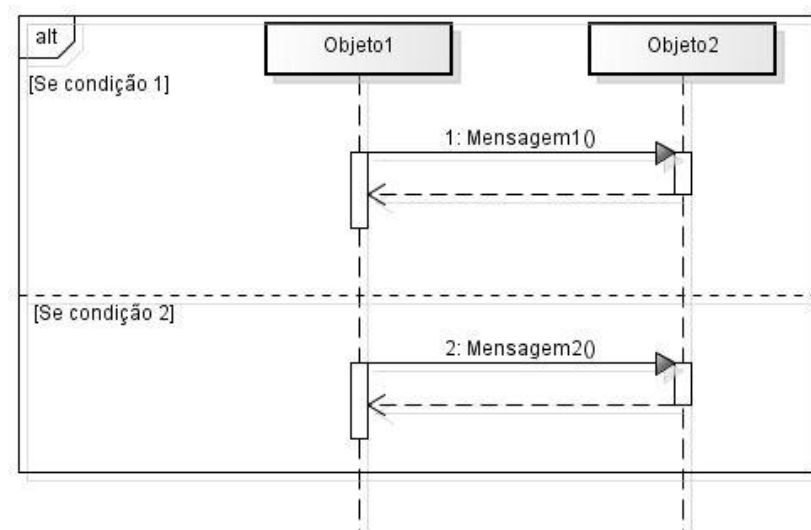


Condição: é representada por um texto delimitado por colchetes.

Fragmentos e Operadores: permitem uma modelagem mais dinâmica, resolvendo problemas como o de laços, testes de alternativas, processamento paralelo, entre outros. São representados por uma janela retangular que apresenta o nome da operação no canto superior esquerdo e as condições entre colchetes. Algumas das operações existentes são:

- Alt (Alternativa): escolha entre duas opções;
- Opt (Opcional): pode ou não ser executado;
- Break (Parar): quebra a execução;
- Loop (Repetição): laço que pode ser repetido;
- Par (Paralelo): execução concorrente;
- Ref (Referência): indica uma operação não exemplificada no momento.

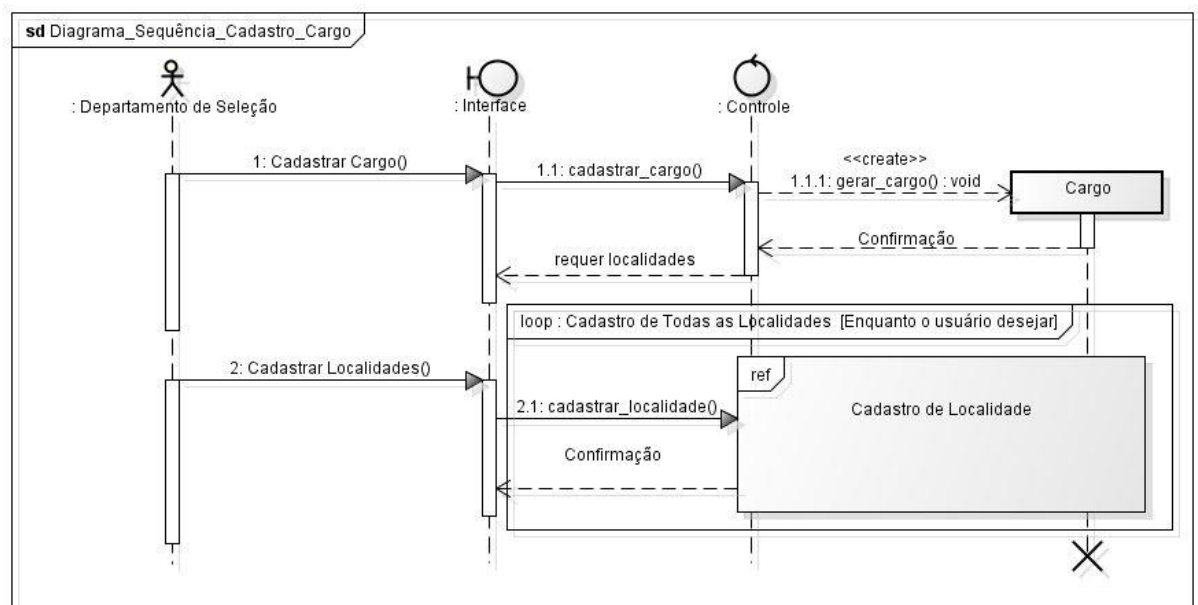
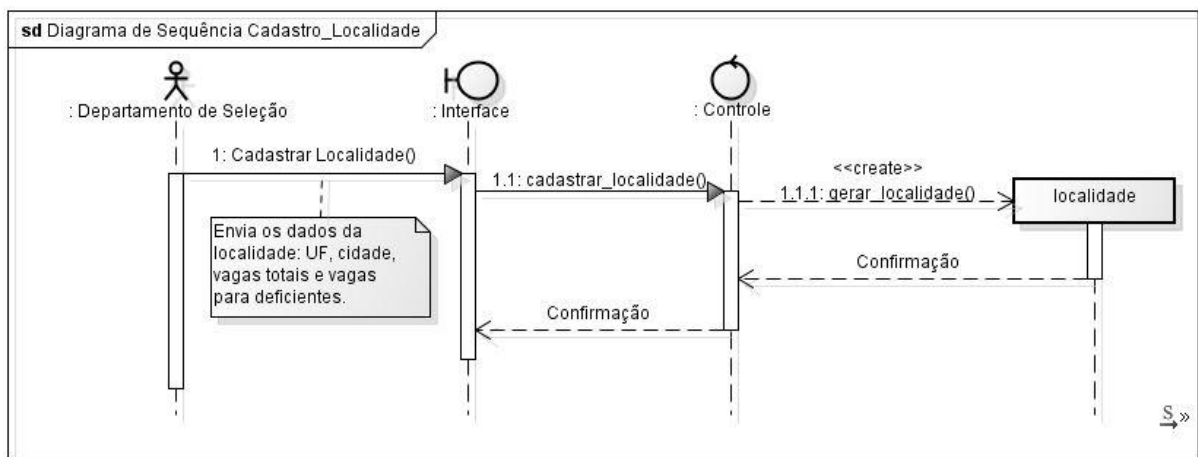
Abaixo temos um exemplo abstrato da utilização do operador "alt", onde apenas uma das duas condições será executada. Para utilizar os outros operadores descritos, basta criar uma janela com a abreviação adequada.



Seguem dois exemplos de Diagramas de Sequência que foram criados a partir de um cenário proposto (Sistema Gerenciador de Processos Seletivos).

A parte do texto que representa a criação do Cadastro Cargo e do Cadastro Localidade foi retirada do texto referente ao sistema:

- "Relação de cargos oferecidos no concurso. Para cada cargo é importante informar:"
- "seu nome (ex: Analista de Sistemas);"
- "as localidades de trabalho, incluindo UF e município;"
- "para cada localidade, número de vagas totais e para deficientes físicos (ex: 5 vagas para a capital do Rio de Janeiro, 10 vagas para Belo Horizonte);"
- (...)
- "critérios de desempate: relação de provas que, na ordem determinada, desempatam o resultado. Em caso de persistir o empate, sabe-se que o critério final prioriza o candidato mais idoso;"
- "o valor da taxa de inscrição."



OBS: No botão Diagrama, apresenta-se o Diagrama de Sequência referente ao cadastro de um candidato no sistema.

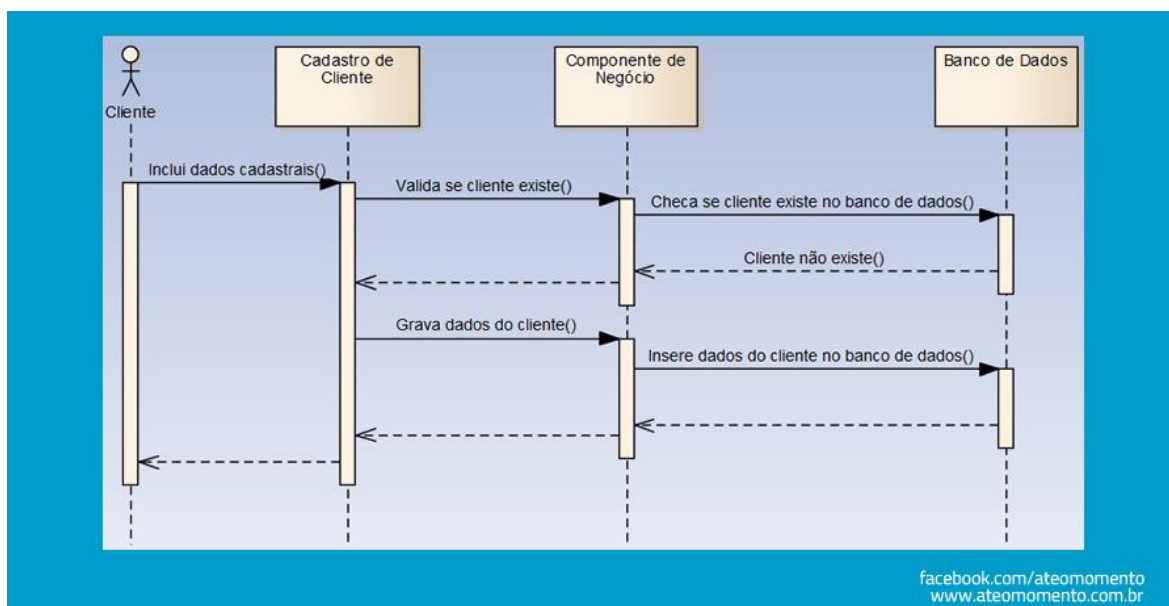
 Boundary	Páginas do lado cliente , HTML, frames, etc. JSP pode ser representado quando contém apenas interface de apresentação.
 Control	Páginas e classes do lado servidor onde está representada a lógica de controle e a lógica da aplicação (o próprio caso de uso)
 Entity	Representa o sistema de dados

Outro Exemplo de Uso

Como citado, um diagrama de sequência tem como finalidade equalizar a comunicação entre profissionais que estão num mesmo contexto técnico (projeto, demanda, produto etc.).

Essa comunicação pode ter como objetivo apenas “esboçar” ideias, mostrar como uma determinada funcionalidade ocorre, como alguma integração ocorre.

Quando a finalidade é esta, utilizamos diagramas de sequência **sem muito rigor** com relação à notação, sem falar em especificação técnica, apenas para fazer o colega de trabalho **entender como alguma funcionalidade “funciona” no sistema**.

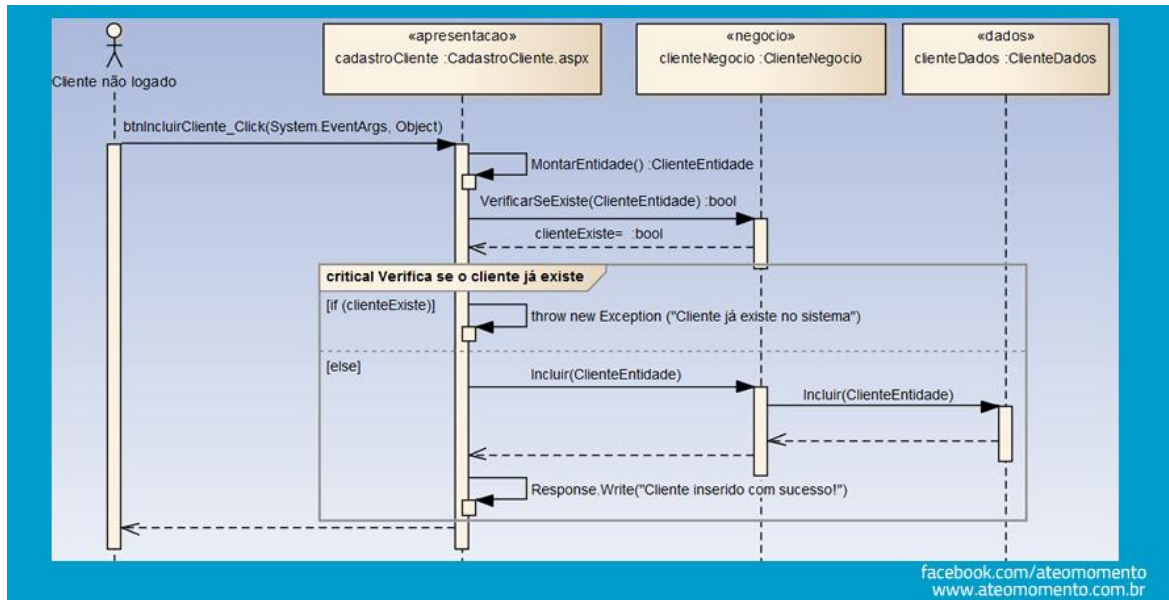


No diagrama acima temos um exemplo do uso com fins de esboço de ideias. Abaixo algumas observações sobre o diagrama acima:

- O Ator “Cliente” é quem inicia o processo. Não há detalhes sobre o elemento.
- Temos três “linhas de vida” (lifelines): Cadastro de Cliente (uma interface gráfica), Componente de Negócio (algo “abstrato” que trata a lógica de negócio) e Banco de Dados (representando o repositório das informações). Não há maiores detalhes sobre estes elementos.
- Há uma mensagem realizada pelo ator na linha de vida “Cadastro de Cliente” que desencadeia outras mensagens nos elementos posteriores, representando todo o fluxo, em nível esboço, da interação originada na inclusão de dados cadastrais do cliente.
- Não há qualquer menção sobre estruturas condicionais, validações etc. afinal, é um diagrama apenas para contextualizar o comportamento de uma funcionalidade, em nível superficial, e não para especificar tecnicamente

esta funcionalidade no menor detalhe de suas premissas/restrições comportamentais.

Abaixo temos outro diagrama de sequência para representar o mesmo contexto, mas não mais com a finalidade de esboço de ideias, e sim, com a finalidade de **especificar tecnicamente** o comportamento da funcionalidade (design), **no menor detalhe**.



No diagrama acima temos um exemplo do uso com fins de projeto (design). Abaixo algumas observações sobre o diagrama acima:

- O Ator “Cliente” é quem inicia o processo. Há detalhes sobre o elemento (não logado).
- Temos três “linhas de vida” (lifelines):
 - CadastroCliente: uma interface gráfica, mas agora uma página .aspx, representada por uma instância sua (instância da classe da página), com estereótipo <<apresentacao>>, o que representa a camada em que a lifeline está, e com as mensagens literais para os métodos contidos no objeto (classe instanciada).
 - ClienteNegocio: uma classe de negócio, representada por uma instância sua (instância da classe, um objeto do tipo), com estereótipo <<negocio>>, o que representa a camada em que a lifeline está, e com as mensagens literais para os métodos contidos no objeto (classe instanciada).
 - ClienteDados: uma classe de acesso a dados, representada por uma instância sua (instância da classe, um objeto do tipo), com estereótipo <<dados>>, o que representa a camada em que a lifeline está, e com as mensagens literais para os métodos contidos no objeto (classe instanciada).
 - Não há menção ao banco de dados, ficando a abstração no diagrama sobre qual é a fonte de dados (neste caso, imagine que isso é transparente para o desenvolvedor, ele só precisa saber qual/como objeto de acesso a dados precisa consumir).